复旦大学 计算机系统基础

Lab1 DataLab 实验报告

整数与浮点数的位级表示及运算

学生姓名	叶梓铭
学号	24301030033
学期	2026 秋季学期
实验环境	Windows + WSL2 + Ubuntu 22.04
编辑器	VS Code WSL
个人仓库	https://github.com/yzm7678024-glitch/ICS2026-DataLab

2026 年 10 月

1 实验目的

本实验完成 P1–P19 共 19 个函数，在限定运算符和操作数的条件下实现整数与 IEEE 754 单精度浮点数的位级运算。通过掩码构造、移位、补码运算和舍入处理，理解数值含义与 32 位编码之间的关系。最终规则检查全部通过，正确性测试得分为 110/110。

整数部分重点练习算术右移与逻辑右移、无分支条件选择，以及在有限位宽下判断大小和处理溢出；浮点部分重点理解规格化数、非规格化数、特殊值和最近偶数舍入。P18、P19 进一步利用分组与逐层合并，实现并行计数和位序反转。

2 实验环境与实验要求

2.1 实验环境

实验在 Windows 主机上通过 WSL2 使用 Ubuntu，VS Code 以 WSL 方式连接 Linux 环境。终端截图中连接名称为 `WSL: Ubuntu-22.04`。在 x86-64 Linux 环境中，使用课程 Makefile 的 32 位目标编译设置完成实验。

项目	实际配置
主机与运行环境	Windows + WSL2 + Ubuntu 22.04
编辑器	VS Code WSL
编译目标	x86-64 Linux 环境中的 32 位 Linux 目标
Makefile 设置	gcc；-O -Wall -fwrapv -m32

2.2 编码规则

按照课程 README 与 `bits.c`，整数题采用 32 位二进制补码模型，有符号右移为算术右移；加法和左移保留低 32 位，移位量必须在 0–31 内。实现只使用函数参数、`int` 局部变量和 0–255 的常量，通用允许运算符为 `! ~ & ^ | + << >>`。P2 进一步限制为仅使用 `~` 和 `&`。

整数题不能使用控制语句、比较运算、乘除法、逻辑与或、类型转换、宏、函数调用或其他数据类型。各函数还必须满足对应的最大操作数；赋值本身不计数。浮点题 P15–P17 可使用 `int`、`unsigned`、任意整数常量和控制语句，但仍禁止真正的浮点类型、浮点常量、浮点运算、类型转换及额外函数。[1][2]

3 测试与最终结果

3.1 运算符与操作数检查

执行 `./check_ops.py bits.c` 后，19 个函数均通过规则检查，最终显示 `All 19 functions passed operator checks.`。图 1 同时给出了各函数的当前操作数与上限；这项检查验证写法合法性，正确性另由 `btest` 检验。

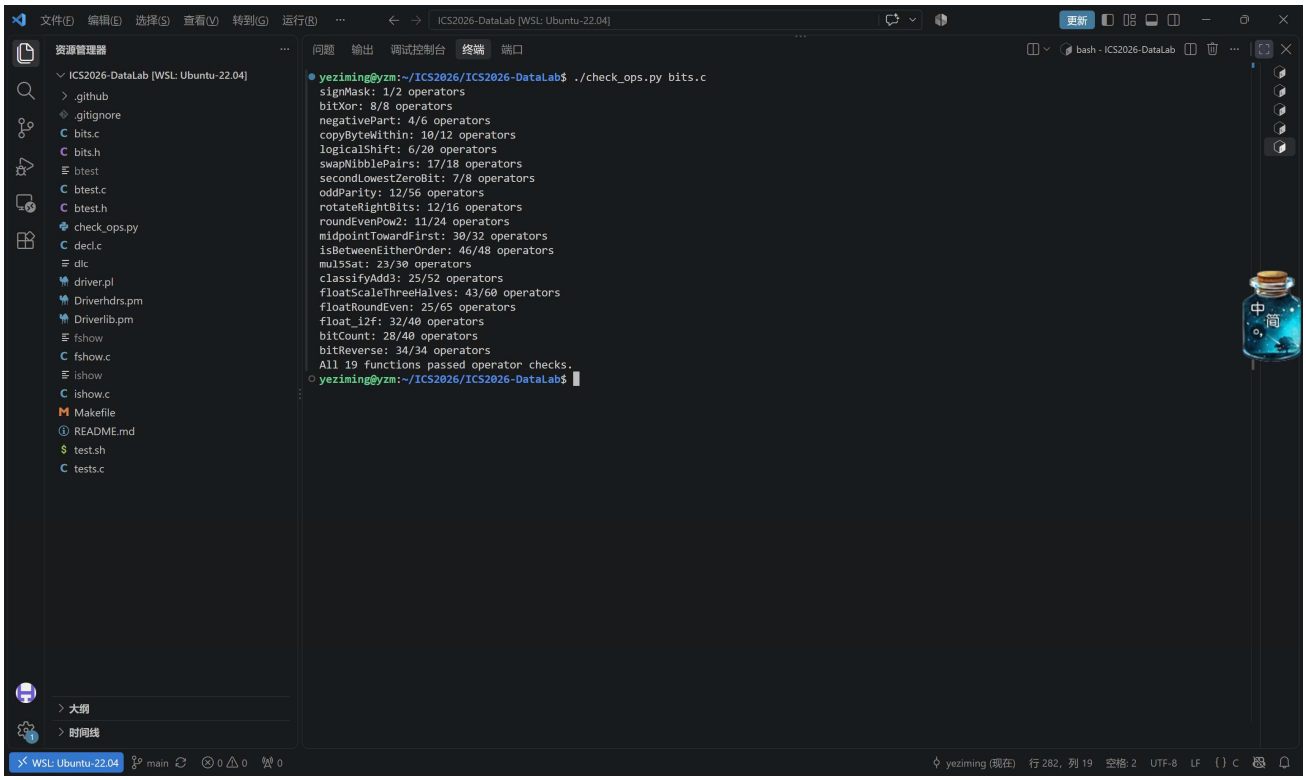


图 1　19 个函数通过运算符与最大操作数检查

图 1 中各题的操作数与上限整理如下。

题号	操作数 / 上限	题号	操作数 / 上限	题号	操作数 / 上限
P1	1/2	P8	12/56	P15	43/60
P2	8/8	P9	12/16	P16	25/65
P3	4/6	P10	11/24	P17	32/40
P4	10/12	P11	30/32	P18	28/40
P5	6/20	P12	46/48	P19	34/34
P6	17/18	P13	23/30		
P7	7/8	P14	25/52		

3.2 正确性与完整流程测试

图 2 为执行 `./btest` 的结果。各函数的 `Errors` 均为 0，最终显示 `Total points: 110/110`，说明最终实现通过课程测试集。

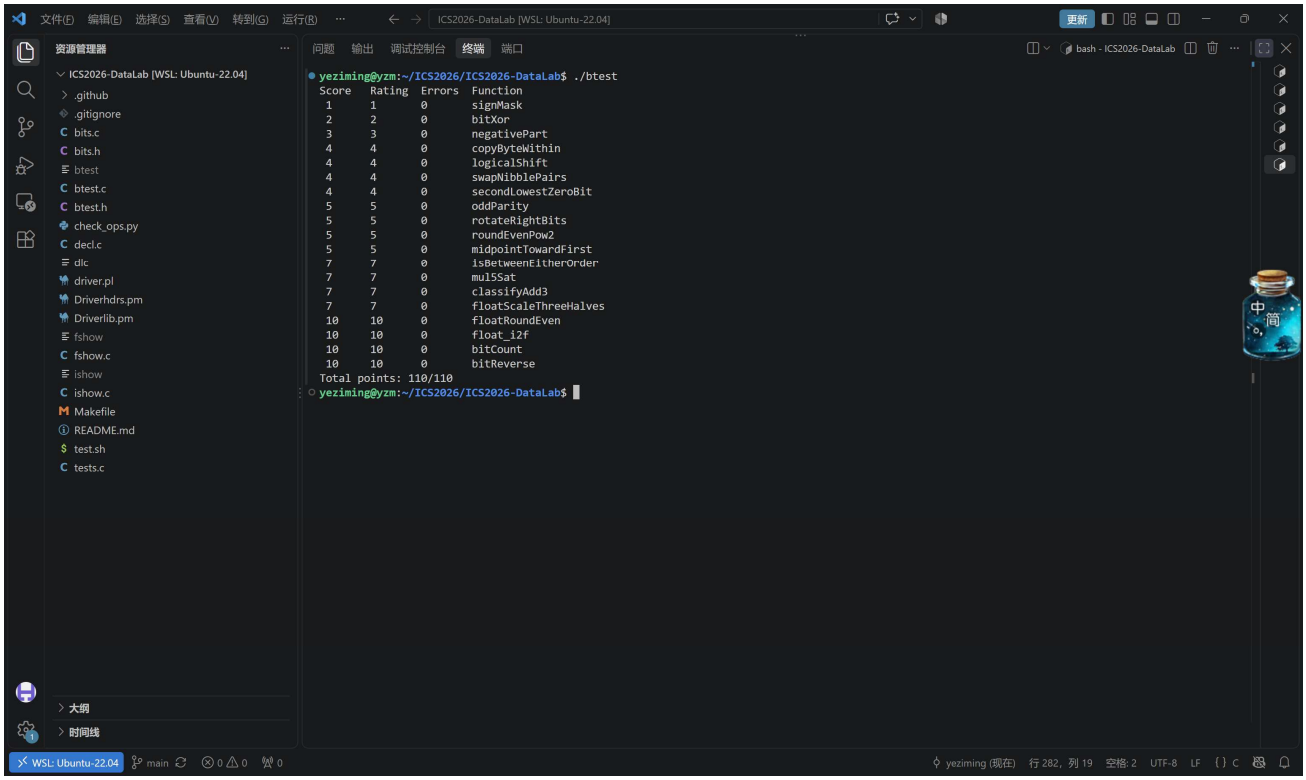


图 2 btest 正确性测试结果 总分 110/110

本地完整流程 `./test.sh` 也通过，最终结果为 `all checks passed: 110/110`。该脚本依次重新构建、检查运算符和操作数、运行正确性测试，只有取得满分时才返回成功。

3.3 GitHub Actions 自动评测

当前 `main` 的最终提交为 `3f42e57`，提交信息为 `Fix rotateRightBits shift range`。图 3 中的 `Autograding Tests #3` 对应该提交，状态为成功，`Autograding report` 显示 `totalPoints: 110`、`maxPoints: 110`，并给出 `Points 110/110`。远程自动评测结果与本地最终结果一致。

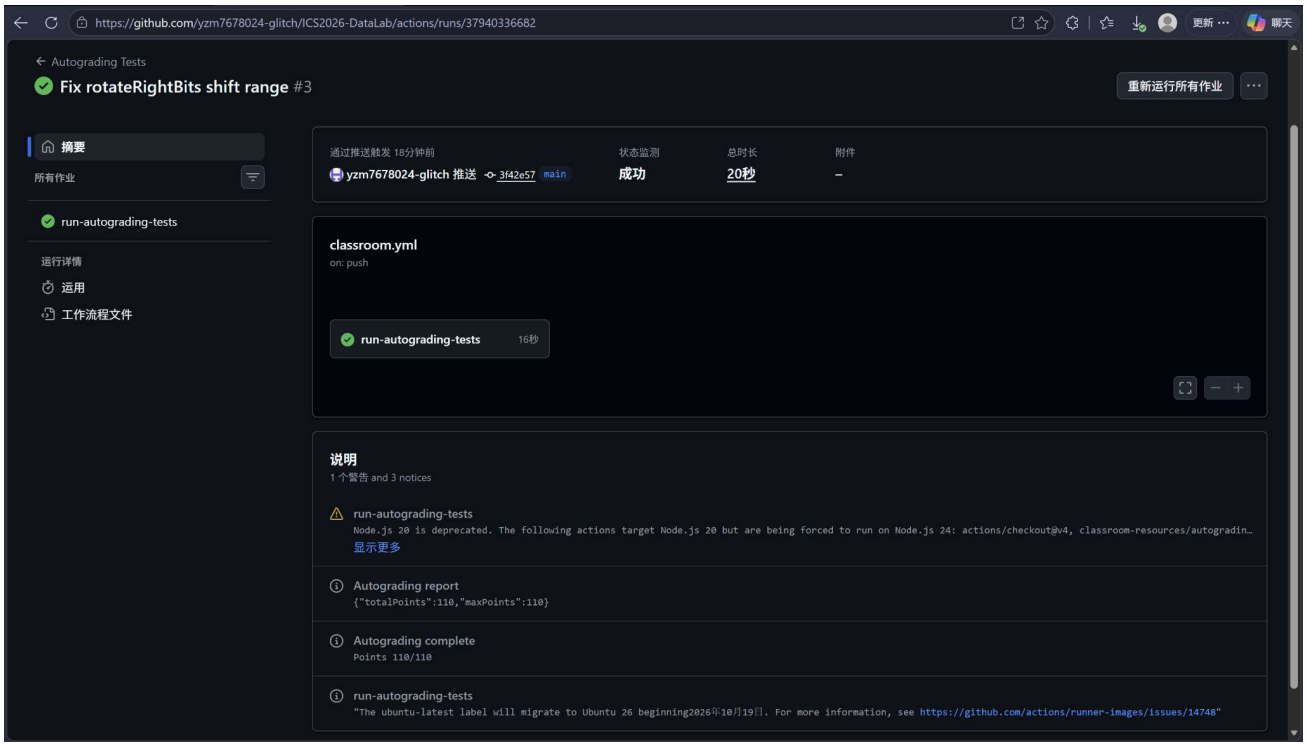


图 3 GitHub Actions 自动评测通过 最终得分 110/110

4 P1 至 P10 实现思路

4.1 基础掩码与局部位操作

P1 signMask

将常量 1 左移 31 位，直接得到仅最高位为 1 的掩码。表达式为 `1<<31`，在实验的 32 位位向量模型中，其位模式为 `0x80000000`。

P2 bitXor

先用 `(~x)&y` 和 `(~y)&x` 分别取出两种“只有一方为 1”的位置，再通过德摩根律，用 `~(~xt&~yt)` 合并两部分。这样实现逐位异或，全程只使用 `~` 和 `&`，操作数为 8/8。

P3 negativePart

用 `x>>31` 生成符号掩码：非负数对应 0，负数对应全 1。将其与补码取负结果 `~x+1` 按位与，即可在负数时保留取负结果，否则返回 0。对 `INT_MIN`，按实验模型保留低 32 位，结果仍为位模式 `0x80000000`。

P4 copyByteWithin

先构造 `255<<(dst<<3)`，将目标字节清零；再把原数右移 `src<<3` 位并与 255，取出源字节。将源字节移到目标位置后与清零结果合并，其他三个字节保持不变。先从原数提取源字节，因此源与目标相同也能正确处理。

P5 logicalShift

先算术右移，再用 `~((1<<31)>>n<<1)` 构造的掩码清除高位补入的符号位。掩码的低 32−n 位为 1，高 n 位为 0。`n=0` 时掩码为全 1，`n=31` 时仅最低位为 1，两端都不需要越界移位。

P6 swapNibblePairs

通过反复左移 8 位并加上 15，构造 `0x0F0F0F0F`，再左移 4 位得到 `0xF0F0F0F0`。分别提取每个字节的低、高半字节，前者左移 4 位，后者右移 4 位并清除算术右移的补位，最后按位或合并，实现每个字节内部的半字节交换。

P7 secondLowestZeroBit

先将输入取反，把寻找 0 转化为寻找 1。随后用 `x&(x+(~0))` 清除最低的 1，再用 `y&(~y+1)` 提取剩余最低的 1，它对应原数第二低的 0。若原数不足两个 0，清除后为 0，最终也返回 0。

P8 oddParity

按 16、8、4、2、1 位依次右移并异或折叠，把原有 32 位的异或结果汇集到最低位。最后返回 `!(x5&1)`，因此原数的 1 个数为偶数时返回 1，为奇数时返回 0。该返回值是补入后使总奇偶性为奇的校验位。

4.2 循环移位与最近偶数舍入

P9 rotateRightBits

最终实现首先执行 `n=n&31`，将任意非负移位量按 32 取模。回卷部分使用 `x<<((~n+1)&31)`，其中左移量是 −n 对 32 取模后的值；右移部分则用与 P5 相同的掩码完成逻辑右移，最后按位或合并。

当取模后的 `n=0` 时，互补左移量也为 0，两部分都保留原数，合并后仍为原数。这样既覆盖 32 的倍数，也保证所有实际移位量始终为 0–31，避免出现左移 32 位。

P10 roundEvenPow2

先用 `q=x>>n` 取得倍数商，`r=1<<(n+(~0))` 取得半程值。代码构造 `t=x+r+(~0)+(q&1)`，再通过 `t>>n<<n` 清除低 n 位，使结果成为目标倍数。

偏置相当于“半程值减 1，再加商的最低位”。余数小于半程时不进位，大于半程时进位；恰好在中点时，偶数商保持，奇数商加 1，因而选择偶数商对应的倍数。题目给定的 x 与 n 范围保证加入偏置后仍在有符号正数范围内。

5 P11 至 P14 整数边界与溢出处理

5.1 中点与区间判断

P11 midpointTowardFirst

用 `mid=(x&y)+((x^y)>>1)` 直接求向下取整的数学中点，避免先计算可能溢出的 `x+y`。其中共同为 1 的部分与异或部分分开处理，除以 2 的效果已融入算术右移。

`(x^y)&1` 判断精确和的奇偶性；和为偶数时，掩码选择 mid。和为奇数时，需要在 mid 与 mid+1 中选择更靠近 x 的整数。代码同号时根据 `x+(~y)+1` 的符号比较，异号时根据 x 的符号比较，只有 x 较大时才加 1，避免异号减法溢出干扰比较。

P12 isBetweenEitherOrder

代码分别构造 x 与 a、x 与 b 是否同号的掩码。同号时用差的符号判断大小，异号时直接用 x 的符号判断，得到 `xa`、`xb` 两个“x 大于等于端点”的 0/1 结果。异号情况下不采用差的符号，避免减法溢出造成错误。

若 x 严格位于两端点之间，两个比较结果不同，`xa^xb` 为 1；区间外则为 0。再用 `!(x^a)` 与 `!(x^b)` 单独识别端点相等，强制返回 1，使区间包含边界，也能处理 a=b 和端点顺序颠倒。

5.2 饱和乘法与三数和分类

P13 mul5Sat

依次计算 `x1=x<<1`、`x2=x1<<1`、`x3=x2+x`，形成 2x、4x、5x 的低 32 位结果。每一阶段都将结果与原始 x 异或并右移 31 位，只要符号发生变化，就产生全 1 的溢出掩码；三个掩码按位或后保留任一阶段发现的溢出。

分阶段判断可以捕捉中间倍增已溢出、后续结果又回到原符号的情况。代码按 x 的符号构造 `INT_MAX` 或 `INT_MIN`，再由溢出掩码选择饱和值；没有溢出时保留 x3。

P14 classifyAdd3

P14 的难点在于不能直接通过 `x+y+z` 判断是否溢出，因为中间结果本身可能已经发生回绕。我先将三个输入的最高位翻转，使原来的有符号数范围转换成便于分析进位的位模式，再依次计算两次 32 位加法产生的最高位进位。

对一次加法，代码利用 `((u&v)|((u|v)&~s))` 的最高位判断是否产生第 32 位进位，并将其化为 0 或 1。两次进位的总数恰好可以区分最终数学和位于 `INT_MIN` 以下、`int` 正常范围内，还是 `INT_MAX` 以上。表中 S 表示最终的精确数学和。

两次进位之和	原精确数学和 S 的范围	返回值
0	$S < -2^{31}$	−1
1	$-2^{31} \le S \le 2^{31}-1$	0
2	$S > 2^{31}-1$	1

因此代码最后用 `carry1+carry2+(~0)` 将两次进位之和减 1，得到 −1、0 或 1，分别表示负溢出、正常范围和正溢出。

6 P15 至 P17 IEEE 754 浮点数实现

6.1 单精度编码与舍入

IEEE 754 单精度编码包含 1 位符号、8 位阶码和 23 位小数域。规格化数的阶码为 1–254，指数偏置为 127，有效数包含隐藏的最高位 1；阶码为 0 时，小数域为 0 表示有符号零，否则表示非规格化数。阶码为 255 时，小数域为 0 表示无穷，否则表示 NaN。[3]

最近偶数舍入先选择距离最近的可表示值；恰好居中时，选择保留部分最低位为 0 的结果。P15、P17 在舍弃有效数低位时应用这一规则，P16 则将数值舍入到最近的整数，并返回该整数的单精度编码。

6.2 三个浮点函数

P15 floatScaleThreeHalves

代码先拆出 `sign=uf&0x80000000`、`exp=(uf>>23)&0xFF`、`frac=uf&0x7FFFFF`。当 exp=255 时直接返回原参数，保留 NaN 编码与无穷；exp=0 时进入非规格化数分支，零也在这个分支中保留原符号。

非规格化数没有隐藏位，因此先计算 `scaled=frac*3`，再取 `q=scaled>>1`。除以 2 被舍去的一位为 1 时恰好在中点，只有 q 为奇数才加 1。若舍入后的 q 达到 `0x800000`，将 exp 设为 1，跨入最小规格化阶码范围，再与符号和小数域重新组合。

规格化数先用 `m=frac|0x800000` 补上隐藏位，再计算 scaled=m*3。若 scaled 达到 `0x2000000`，有效数乘 3/2 后需要升阶，代码先将 exp 加 1，再右移 2 位并以 `scaled&3` 取得余数：余数大于 2，或等于 2 且 q 为奇数时进位。否则右移 1 位，用与非规格化分支相同的中点判断。

完成舍入后，若 q 达到 `0x1000000`，再右移 1 位并增加阶码，以处理舍入导致的有效数进位。最终 exp 达到 255 时返回相应符号的无穷，否则重新拼接 sign、exp 和 q 的低 23 位。

P16 floatRoundEven

代码按阶码分段处理。exp≥150 时，有限数的实际指数至少为 23，已无小数部分；NaN 和无穷也满足该条件，因此都直接返回原编码。exp<126 时绝对值小于 0.5，返回 sign，使负数舍入后仍为 −0。

exp=126 对应绝对值在 0.5 到 1 之间：frac=0 时恰好为 0.5，选择偶整数 0；frac 非零时选择 1，分别通过 sign 或 `sign|0x3F800000` 返回。

其余阶码为 127–149，令 `shift=150-exp`，构造低 shift 位掩码并取得 `rest=frac&mask`。先清除这些小数位；若 rest 大于半程，或等于半程且 `((mant>>shift)&1)` 为 1，则对编码加上 `1u<<shift`。mant 包含隐藏位，因而能正确判断保留整数部分的奇偶性；编码加法也会自然处理向阶码的进位。

P17 float_i2f

x=0 时直接返回 0。其他输入先赋给无符号变量 absx；负数设置 sign 后，用 `~absx+1` 取得幅值。取负在无符号域进行，所以 `INT_MIN` 可得到幅值 `0x80000000`，不会依赖有符号绝对值。

从 pos=31 开始向下寻找最高有效位 1，令 exp=pos+127。若 pos≤23，把幅值左移到有效数位置并取低 23 位，整数可以精确表示；若 pos>23，则令 shift=pos−23，保留最高 24 个有效位，并分别取出被舍弃的 remain 与半程值 half。

remain>half 时进位；remain=half 时仅在保留小数域最低位为 1 时进位，实现最近偶数舍入。若 frac 加 1 后达到 `0x800000`，将其清零并使 exp 加 1，处理有效数进位后的重新规格化。最后拼接 sign、exp、frac。

7 P18 至 P19 并行位运算

7.1 并行计数

P18 bitCount

先把 8 位常量 `0x55` 复制为 16 位，再复制成 `0x55555555`。用 `(mask1&x)+((x>>1)&mask1)` 在每 2 位内计算 1 的个数。之后按相同方式构造 `0x33333333`，将相邻的两组 2 位计数合并为 4 位计数。

构造 `0x0F0F0F0F` 后，使用 `(x+(x>>4))&mask3` 合并为每字节计数。随后直接执行 `x=x+(x>>8)` 和 `x=x+(x>>16)`，将计数继续合并到 16 位、32 位；这些阶段只需要最后的低位结果，无需逐层新建掩码。最终用 `x&0x3F` 取得总计数，范围为 0–32。

7.2 分治交换

P19 bitReverse

代码按 16、8、4、2、1 位的顺序逐层交换相邻两组位。第一步构造 `0x0000FFFF`，通过 `(x<<16)+((x>>16)&mask1)` 交换高、低半字；左移部分的低 16 位天然为 0，因此无需再与掩码。

后续掩码由前一个掩码移位并异或生成：依次得到 `0x00FF00FF`、`0x0F0F0F0F`、`0x33333333` 和 `0x55555555`。每层把一侧掩出后左移，另一侧右移后掩出，再用加法合并；两部分占据互不重叠的位，所以合并不会产生额外进位。五层交换完成后，原 bit i 映射到 bit 31−i。

这种掩码复用减少了从头构造常量的操作。最终规则检查为 34/34，操作数恰好达到题目上限。

8 实验总结

这次 DataLab 和我之前写过的普通 C 程序很不一样。随着一道道题做下来，我逐渐习惯了从 32 位二进制的角度思考问题。算术右移、补码、掩码、溢出这些原本只在课上理解的概念，也开始变成真正可以用来解决问题的工具。

这段时间与 DataLab 的“搏斗”让我收获很多。实验把许多原本在 C 语言中习以为常的操作都限制甚至禁用了，一开始确实很难受，但随着逐渐适应，32 位的 bit 在脑海中也变得越来越熟悉，仿佛可以直接去观察和操控它们。这也是我第一次很明显地感受到，自己确实是在实践中学会了很多东西。随着题目难度不断增加，每一道题都在打破我原来的编程习惯：整数发生回绕时应该怎样理解，不能使用 `if` 时怎样用掩码实现条件选择，怎样只利用加法、移位和异或完成原本很自然的操作，什么时候需要考虑溢出，又有哪些特殊边界。甚至连判断两个数是否相等这样平时一个运算符就能完成的事情，在操作数受限时都需要重新设计。这些限制反而让我不得不认真思考每一步运算在二进制层面到底发生了什么。

另外，P9 给我的印象也很深。最初的实现已经能够通过 `btest` 和 GitHub Actions，但后来重新检查题目的输入范围时，我发现原来的代码没有严格处理大于 31 的移位量。修正为按 32 取模后重新测试，最终结果仍然是 110/110。这个过程让我意识到，测试通过并不一定代表实现已经完全符合题目规格，尤其是位运算、移位和溢出这类底层操作，还需要主动回头检查边界条件和语言规则。

总的来说，非常感谢这次的 lab，感谢助教老师对题目的精心设计。

参考资料

[1] 复旦大学计算机系统基础 2026 秋季学期，Lab1 DataLab 官方实验页面。
https://ics-26fall-fdu.github.io/labs/lab1-data-lab/

[2] ICS-26Fall-FDU/DataLab，官方模板仓库 README.md 与 bits.c。
https://github.com/ICS-26Fall-FDU/DataLab/blob/main/README.md
https://github.com/ICS-26Fall-FDU/DataLab/blob/main/bits.c

[3] Bryant, R. E. & O’Hallaron, D. R. Computer Systems: A Programmer’s Perspective. Chapter 2, Representing and Manipulating Information.